

Basics of Deep Learning

Introduction

Christopher Lee

Who am I and what do I work on?

- Education:
 - B.S. Comp Sci at Illinois Institute of Technology
 - M.S. Comp Sci at University of Hawaii Manoa
 - Currently in PHD program at University of Hawaii
- Hobbies:
 - Photography, Leatherworking, Game Dev, Travel
- What I work on for Sage:
 - Anomaly Detection and LLM



Introduction to Deep Learning

Subfield of machine learning that utilizes deep neural networks to learn patterns.

Mindset to Approach Deep Learning

- We are trying to create a pattern recognition system.
- The more examples of the pattern we have, the more uncertainty gets reduced.
- Because our system is probabilistic, It doesn't give the absolute truth, but the best guess under uncertainty.
- Think of the predictions as a distribution
- How to frame your problem: regression, classification, clustering, etc.

Table of Contents

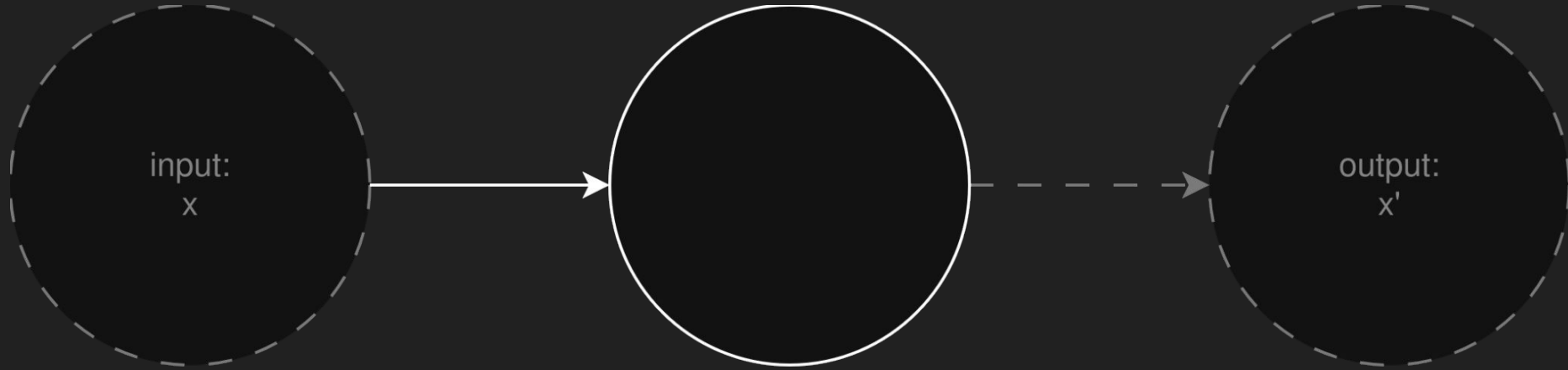
1. Anatomy of Artificial Neuron and Computing a Forward Pass
2. Dataset Properties
3. Training a Neural Network

Neural Network

Fundamental Understanding

- Whats a neuron?
- How and why does it work?
- Common terminology

High Level Idea: It's Just Another Math Function



x inputs/features



some function operation
here... think $x' = z(x)$ where z is
a function like $z(x) = x + 10$

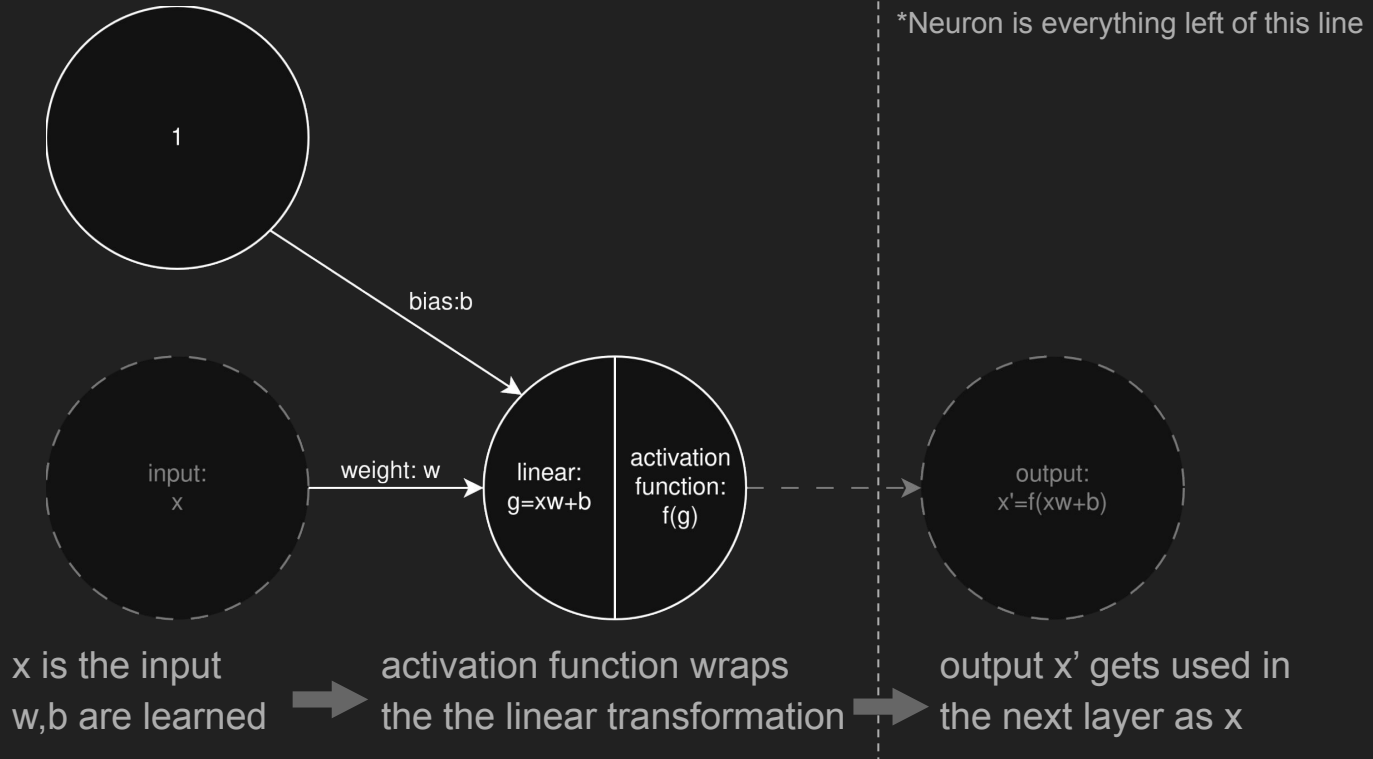


output/prediction is x'

Some notations have x' as $\hat{y}$

Anatomy of a Neuron with **Single Input**

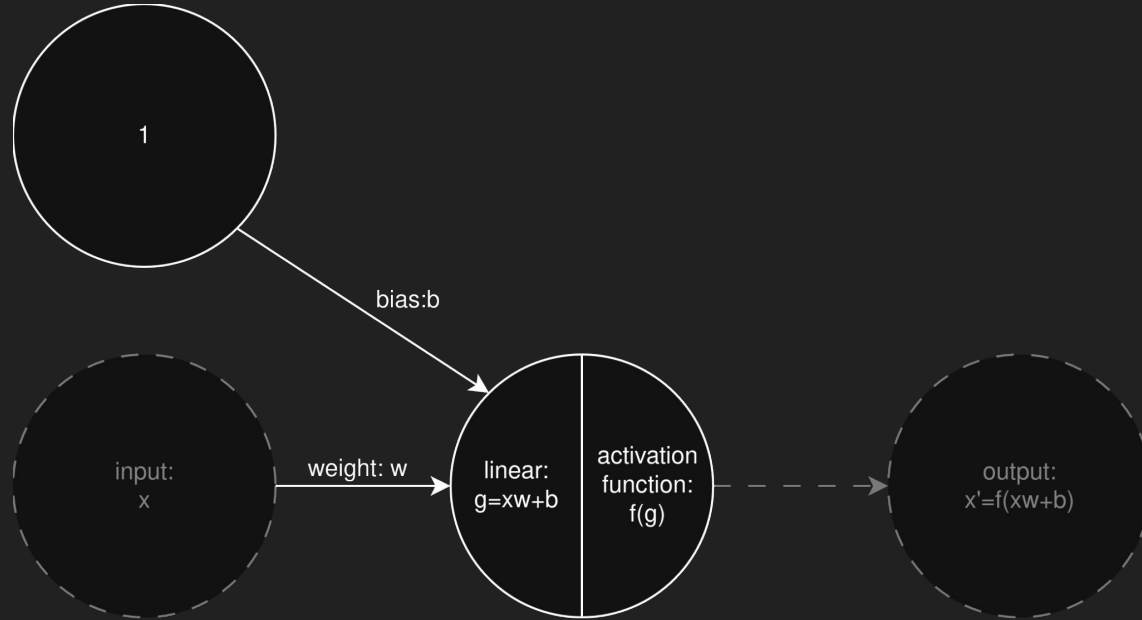
Terms	
Input	x
weight	w
bias	b
activation	$f(g)$



Forward pass for Neuron

When we compute from input to output, we call it a **forward pass**.

During training (later on) we will introduce **backpropagation**, which is going backwards



$$g=xw+b$$



Then apply activation function, we use ReLU as an example here.
 $\text{ReLU}(g) = \max(0, g)$



$$x' = \max(0, xw+b)$$

Anatomy of a Neuron with Two Inputs

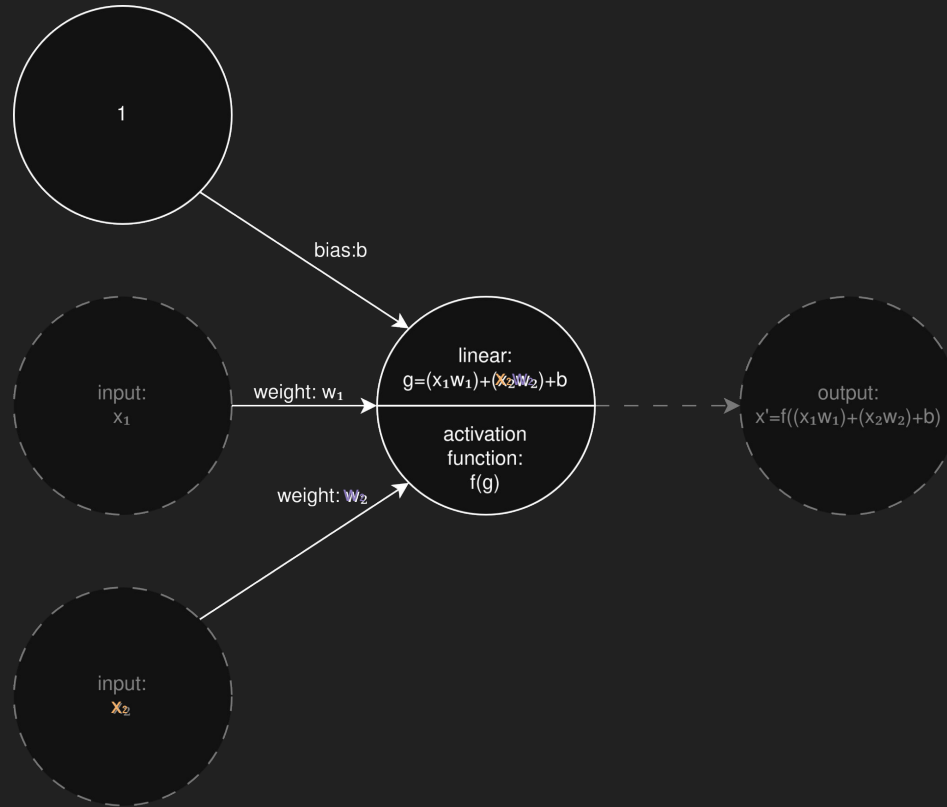
When we add another input, we introduce another **input**, **weight** pair.

Now the linear combination goes from (1 input)

$$g = x_1 w_1 + b$$

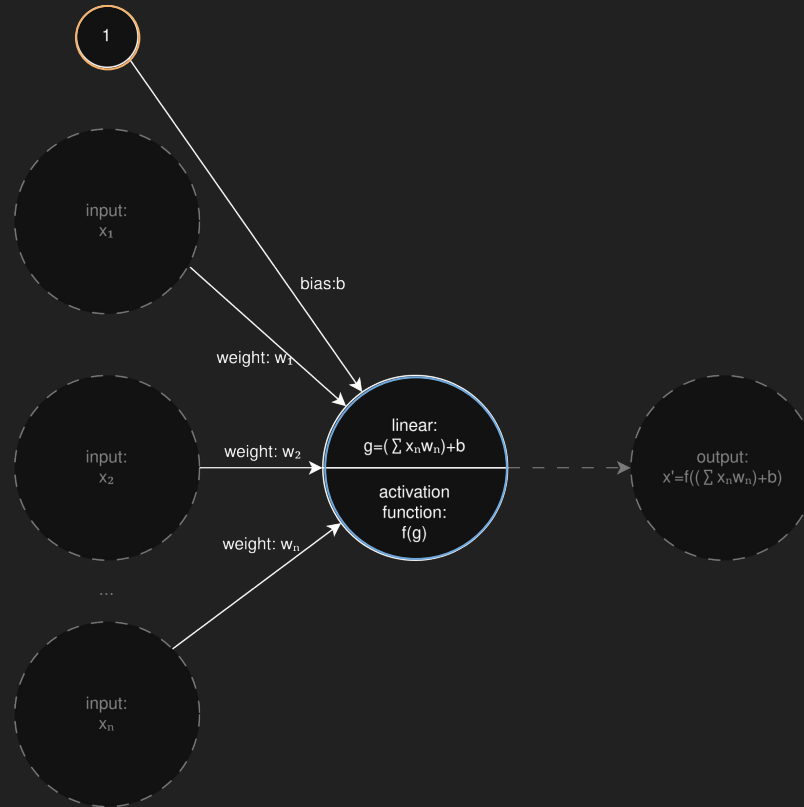
To (2 inputs)

$$g = x_1 w_1 + x_2 w_2 + b$$



Anatomy of a Neuron with N Inputs

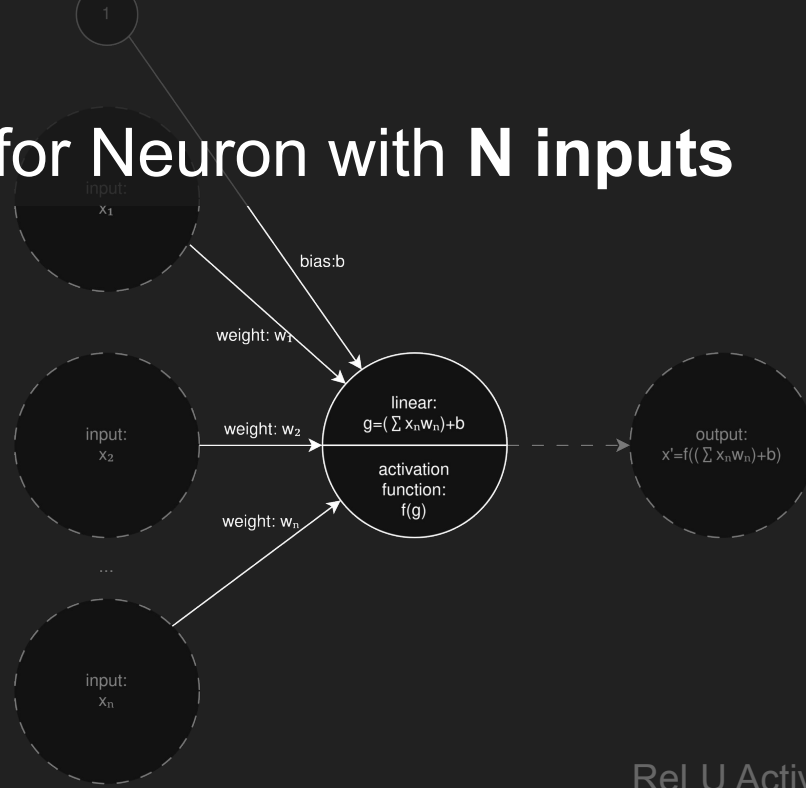
The bias term is **one bias** per **neuron**, so if we have 2 middle neurons, there would be 2 bias neurons



Also, when we compute from input to output, we call it a **forward pass**.

During training (later on) we will introduce **backpropagation**

Forward pass for Neuron with N inputs



ReLU Activation: $\max(0, g)$

$$g = x_1 w_1 + x_2 w_2 + \dots + x_n w_n + b$$



$$x' = \max(0, g)$$

$$x' = \max(0, x_1 w_1 + x_2 w_2 + \dots + x_n w_n + b)$$

Anatomy of a 2 Layer Network

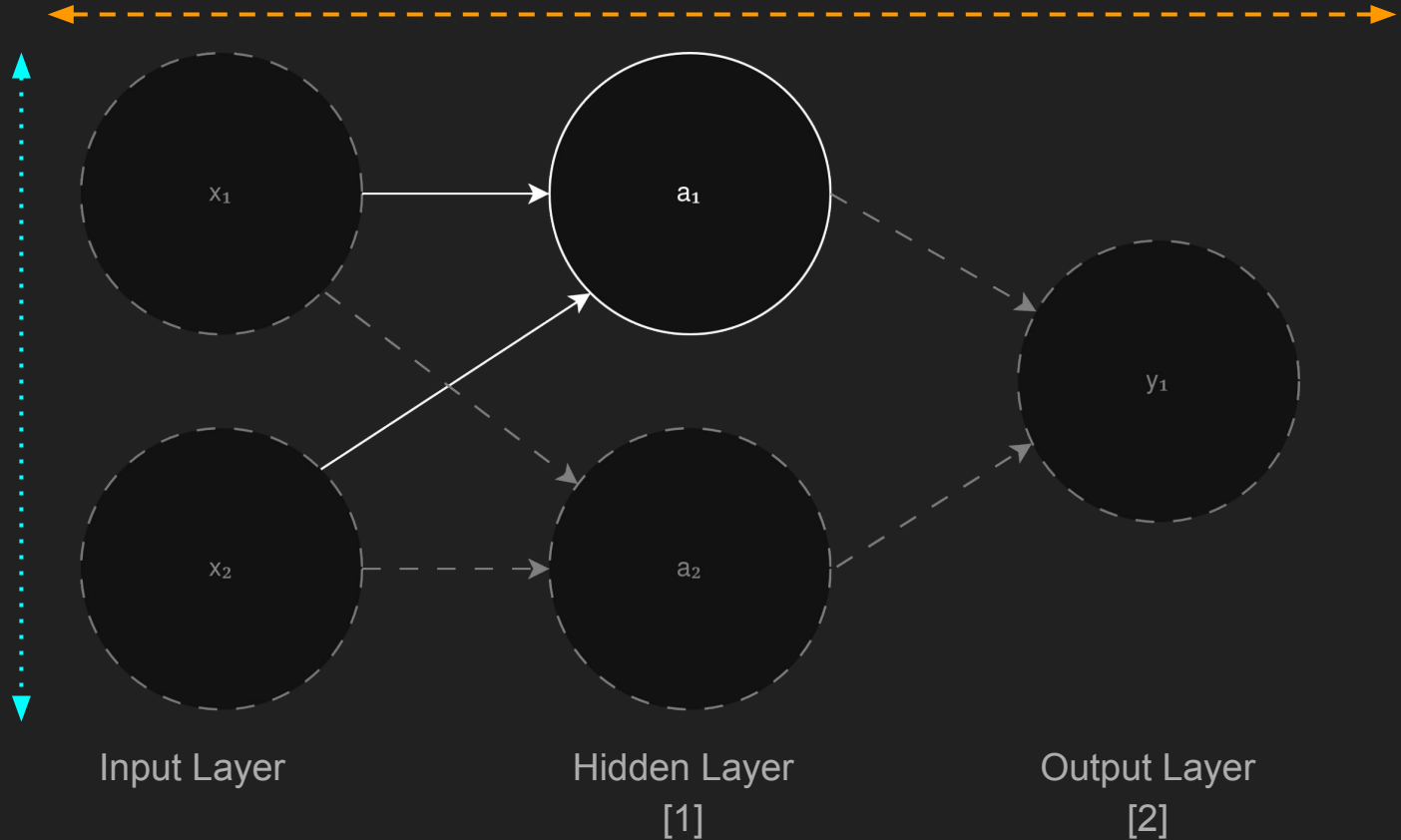
Terms	
Depth	← - - - - - →
Width (number per layer)	↑ : ↓

Depth: 2

(input layer doesn't count)

Layer 1 Width: 2

Layer 2 Width: 1



*(1dim = 1feature = 1axis)

Example: [sqft, num_garage, bedrooms] (3 features)

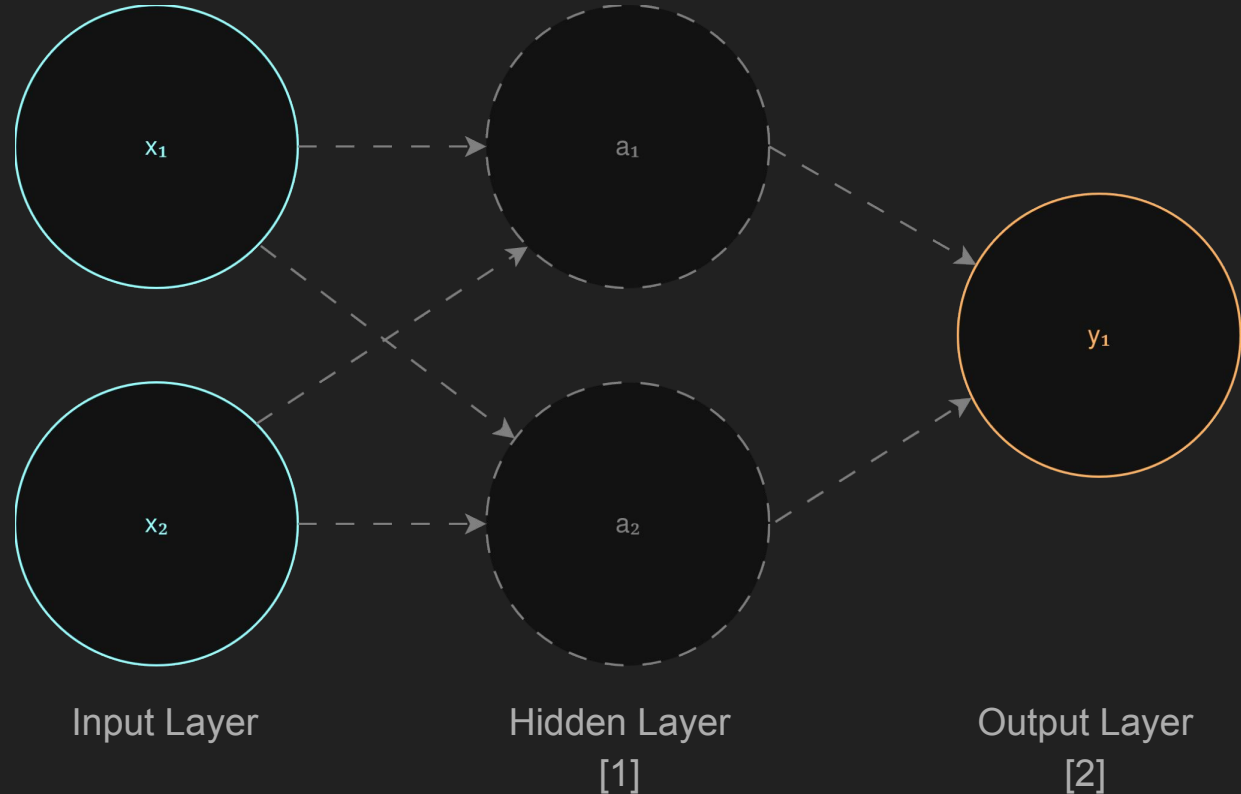
Anatomy of a 2 Layer Network

Terms	
Output, Prediction	
Input, Features, Dimensionality	

Inputs: 2 dims (dimensions)*

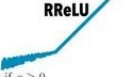
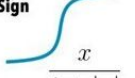
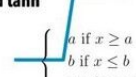
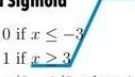
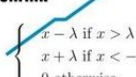
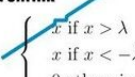
Outputs: 1

This means that given 2 numbers, x_1 , and x_2 we can predict one result y_1



Activation Functions: Hidden Layer

Neural Network Activation Functions: a small subset!

ReLU  $\max(0, x)$	GELU  $\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$	PReLU  $\max(0, x)$
ELU  $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish  $\frac{x}{1 + \exp -x}$	SELU  $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus  $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish  $x \tanh \left(\frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$	RReLU  $\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$
HardSwish  $\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid  $\frac{1}{1 + \exp(-x)}$	SoftSign  $\frac{x}{1 + x }$
Tanh  $\tanh(x)$	Hard tanh  $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid  $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink  $x - \tanh(x)$	Soft Shrink  $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink  $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

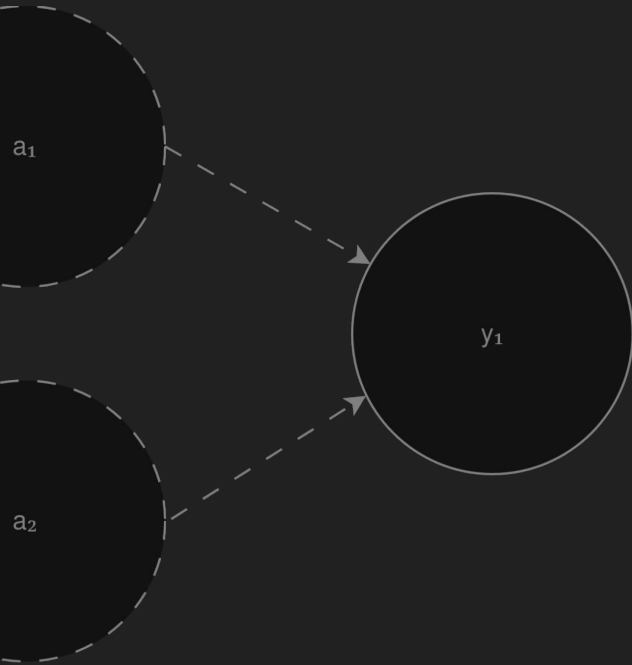
For the hidden layers (middle layers, not output or inputs) it is **standard** to use **ReLU**. If the system is not training (could be from dying neurons), try **Leaky ReLU**, **ELU**, or **Swish**.

Try to avoid non-conventional activations such as **Sigmoid** for hidden layers as they can introduce problems like vanishing gradients for deep networks.

What it looks like in Pytorch

```
self.head = nn.Sequential(  
    nn.Linear(embed_dim, hidden_dim), # linear:  $w \cdot x + b$   
    nn.ReLU(),                        # relu:  activation()  
                                     # -----  
                                     # activation(  $w \cdot x + b$  )  
    nn.Linear(hidden_dim, num_classes),  
)
```

Activation Functions: Regression Output Layer

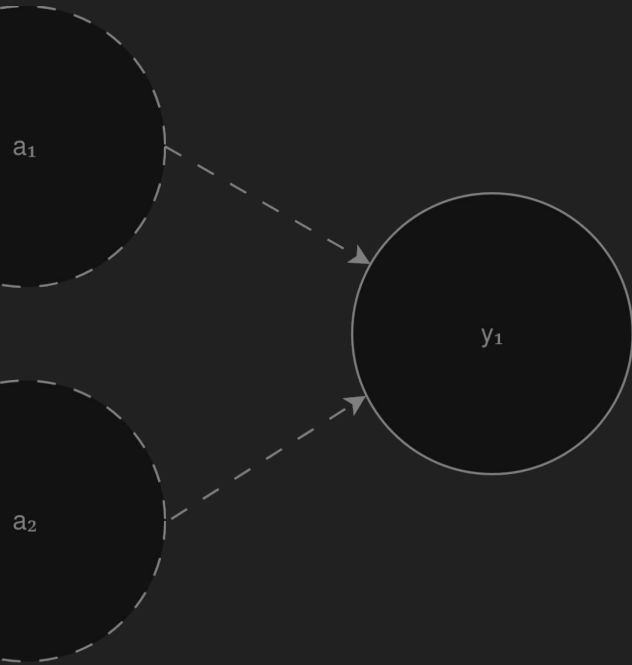


Regression tasks are predicting answers like price, number of burgers, count of cars, etc. They are continuous (not discrete) scores.

Rule of thumb is to pick an activation function based on bounds.

Activations	Bounds	Example Use Case
None/ Linear	$[-\infty, \infty]$	Price, temperature
ReLU	$[0, \infty]$	Items sold
Tanh	$[-1, 1]$	Sentiment scores
...	...	...

Activation Functions: Classification Output Layer



Classification tasks normally want probabilistic outputs, they are typically discrete answers that we assign a class/label to.

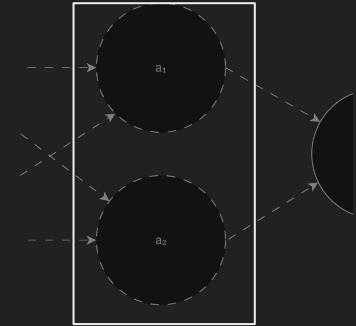
Logits are raw (unnormalized) predictions of the **output/final layer** before the **activation function** is applied. They are typically used with the intent to convert to a probabilistic output.

Activation Function Selection:

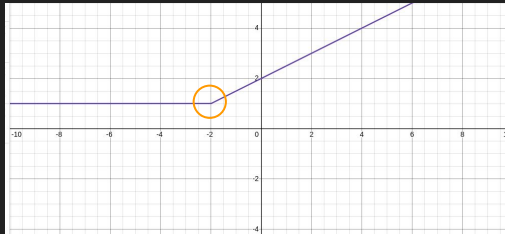
- For 1 width outputs, we typically use **sigmoid** (activation function) to bound our results to $[0,1]$
- For 2 or more width outputs, we use typically use **softmax** (activation function) to normalize the sum of all logits to 1.0 (100%) since we want a probability distribution.

Universal Approximation Theorem

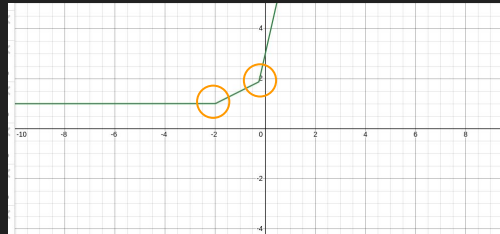
- “The universal approximation theorems (UATs) state that neural networks with a certain structure can, in principle, **approximate** any **continuous function** to any desired degree of accuracy.”^[1]
- Every neuron we add, in the **hidden layer**, can introduce a **bend** (or kink) in the function.
 - What kinds of bends or kinks are determined by the **activation function**
 - Trade-off is overfitting and poor generalization (we will talk about this in training section)



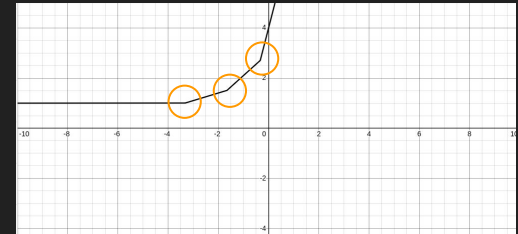
Hidden layer



1 Neuron (w/ ReLU)



2 Neurons (w/ ReLU)



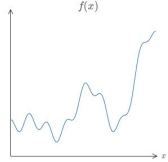
3 Neurons (w/ ReLU)

Universal Approximation Theorem

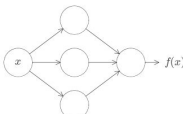
- More Resources and Examples

CHAPTER 4
A visual proof that neural nets can compute any function

One of the most striking facts about neural networks is that they can compute any function at all. That is, suppose someone hands you some complicated, wiggly function, $f(x)$:



No matter what the function, there is guaranteed to be a neural network so that for every possible input, x , the value $f(x)$ (or some close approximation) is output from the network, e.g.:



Neural Networks and Deep Learning
What this book is about
On the exercises and problems
► Using neural nets to recognize handwritten digits
► How the backpropagation algorithm works
► Improving the way neural networks learn
► A visual proof that neural nets can compute any function
► Why are deep neural networks hard to train?
► Deep learning
Appendix: Is there a simple algorithm for intelligence?
Acknowledgements
Frequently Asked Questions

If you benefit from the book, please make a small donation. I suggest \$5, but you can choose the amount.

Donate

Alternatively, you can make a donation by sending me Bitcoin, at address
1B6WJ5E5ED5AwfN45P9W6CQq7K5SAA

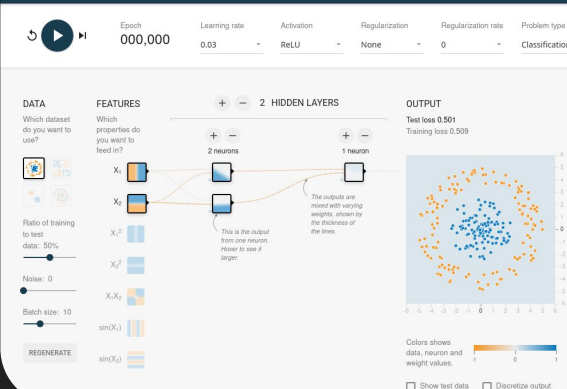
Sponsors
 Lambda

In depth explanation and demos:
<http://neuralnetworksanddeeplearning.com/chap4.html>

Tinker With a **Neural Network** in Your Browser.
Don't Worry, You Can't Break It. We Promise.

Epoch: 000,000 Learning rate: 0.03 Activation: ReLU Regularization: None Regularization rate: 0 Problem type: Classification

DATA: Which dataset do you want to use?
FEATURES: Which properties do you want to feed in?
HIDDEN LAYERS: 2 HIDDEN LAYERS (2 neurons, 1 neuron)
OUTPUT: Test loss 0.501 Training loss 0.509



Colors shows data, neurons and weight values.
Show test data Discretize output

2 dimensional data example:
<https://playground.tensorflow.org/>

Why not make the network infinitely wide?

Deeper networks can be better at...

- Generalization
- Scalability/ Easier to train: fewer neurons needed to achieve same task
- Feature reuse: able to reuse information from prior computation in previous layer

Deeper networks can be harder to train, so striking a balance (experimentation, trial and error, experience) is part of the process.

Transfer of Information

- The network cannot effectively predict what it doesn't have information about.
 - The **inputs/features** are the information given to the network.
 - The important aspects of the inputs has to flow through the network (they get transformed, and filtered by the neurons).
 - Some networks enforce ideas like invariance to guarantee or encourage certain behaviours.
 - If important features are lost, the model can have difficulties learning.
 - If **irrelevant** information and **all relevant information** is given, model's learning can be degraded.
 - Noisy information can be hard to learn, try to preprocess/ clean up the information first.
- Rule of thumb (another perspective): if a human can learn it, there's a good chance the network can learn it.

High Level Example:

Information (inputs): 3 oranges and 5 apples

Question: More apples than oranges?

Answer (output): Yes; 5 apples > 3 oranges

Information (inputs): 3 bananas and 5 apples

Question: More apples than oranges?

Answer (output): Maybe?

Key takeaway from this section

- Neurons are used to create functions that can separate the data, predict patterns, etc. given that we have the right weights and bias values.
- We can stack more neurons to approximate functions more accurately (but can cause overfitting, we talk about this later)
- The network need relevant features/inputs (information)
- Terminologies used to effectively communicate to an AI coding agent

Other Types of Network Network Architectures

More practical stuff

Other Types of Neural Networks

A mostly complete chart of Neural Networks

©2019 Fjodor van Veen & Stefan Leijnen asimovinstitute.org

- Input Cell
- Backfed Input Cell
- △ Noisy Input Cell
- Hidden Cell
- Probabilistic Hidden Cell
- △ Spiking Hidden Cell
- Capsule Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- △ Gated Memory Cell
- Kernel
- Convolution or Pool

Perceptron (P)



Feed Forward (FF)



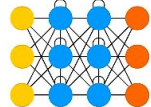
Radial Basis Network (RBF)



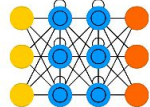
Deep Feed Forward (DFF)



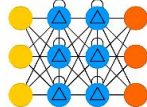
Recurrent Neural Network (RNN)



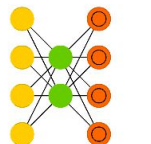
Long / Short Term Memory (LSTM)



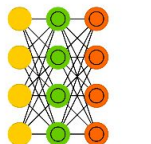
Gated Recurrent Unit (GRU)



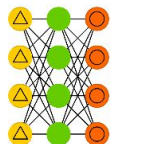
Auto Encoder (AE)



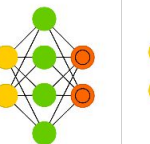
Variational AE (VAE)



Denosing AE (DAE)



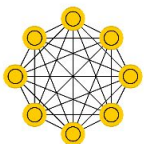
Sparse AE (SAE)



Markov Chain (MC)



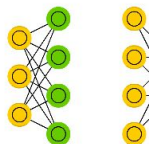
Hopfield Network (HN)



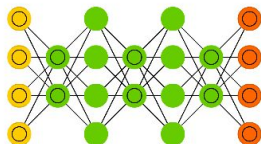
Boltzmann Machine (BM)



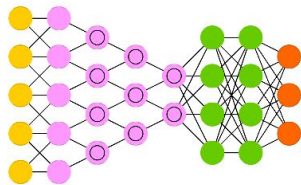
Restricted BM (RBM)



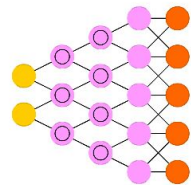
Deep Belief Network (DBN)



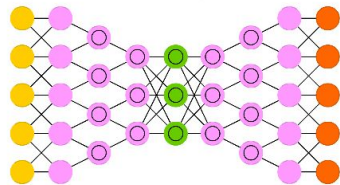
Deep Convolutional Network (DCN)



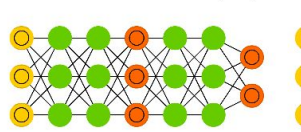
Deconvolutional Network (DN)



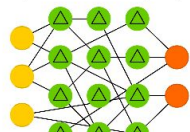
Deep Convolutional Inverse Graphics Network (DCIGN)



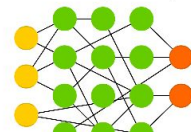
Generative Adversarial Network (GAN)



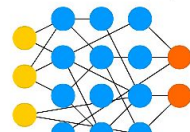
Liquid State Machine (LSM)



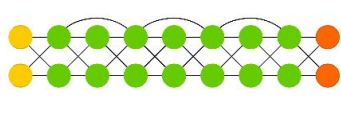
Extreme Learning Machine (ELM)



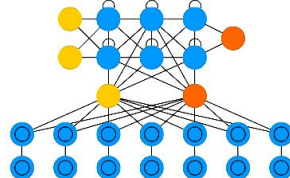
Echo State Network (ESN)



Deep Residual Network (DRN)



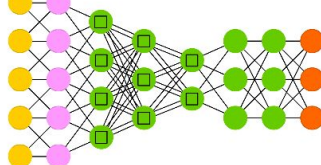
Differentiable Neural Computer (DNC)



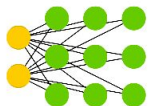
Neural Turing Machine (NTM)



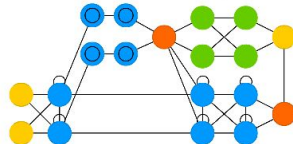
Capsule Network (CN)



Kohonen Network (KN)



Attention Network (AN)

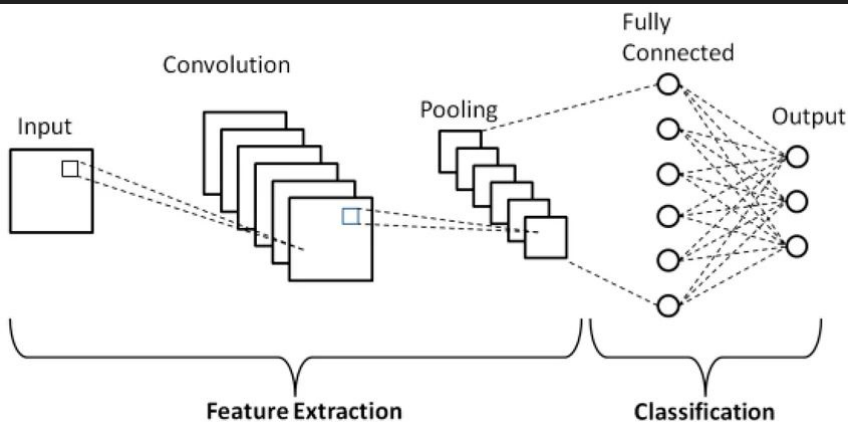


Why so many? Certain networks are more optimal for certain tasks.

<https://www.asimovinstitute.org/neural-network-zoo/>

A Clearer Example of a Two Types of Networks

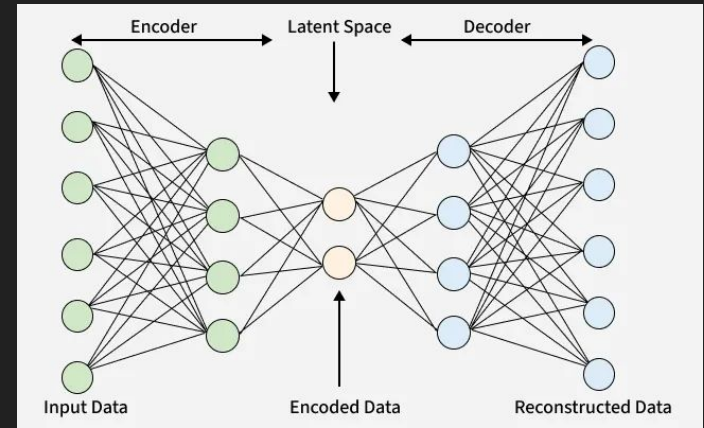
CNN



For CNNs, a kernel is trained, this kernel shifts around the image. They are good at exploiting spatial localities (relating nearby pixels to each other)

https://www.researchgate.net/figure/Schematic-diagram-of-a-basic-convolutional-neural-network-CNN-architecture-26_fig1_336805909

AutoEncoders



Autoencoders force the network to learn only the important parts with the bottleneck layer (middle layer)

<https://www.geeksforgeeks.org/deep-learning/types-of-autoencoders/>

Datasets

Curation and Preprocessing

Dataset Curation

- What should our dataset embody?
 - Relevant to Task
 - Data should pertain to the task, if we train a cat detector, we don't want to detect dogs.
 - Sufficient Quantity of Examples
 - Depending on technique, you may need lots of data, in the magnitude of thousands
 - Diversity
 - Data should capture the entire distribution, including low probabilistic events
 - Accurately Labeled
 - Dataset should be annotated/labeled properly
 - Representative/ Balanced
 - Depending on task, we may want the data to reflect the distribution or we may want to balance the dataset.
 - No Contamination
 - training/validation/test sets should not have exact duplicates of data

Dataset Anatomy

Examples are combinations of features/inputs and **targets/labels**

Usually we have multiple features/inputs used to predict the **targets/labels**

Sometimes we can have multiple **targets/labels** as well, it just depends on task



features/inputs must correlate to the **targets/labels**. As an example:

Features [n_wheels, n_seats]	Target [vehicle_type]
[4,4]	[car]
[2,1]	[motorcycle]
[4,6]	[van]

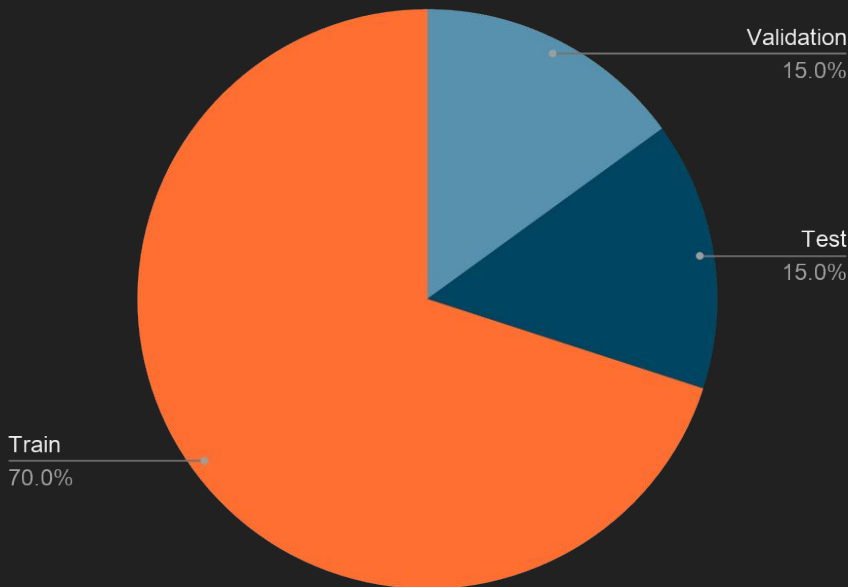
Data Splits

Train: data used to train the model weights

Validate: Measure performance and tuning decisions based on results, we call it hyperparameter tuning

Test: Performance expected in the real use case. **DO NOT MAKE DECISIONS OFF OF THIS SET!**

Dataset Splits



We typically want the splits to be stratified. Meaning that each data split maintains the same ratios.

Fraud Example:
Train: 90% non-fraud/10% fraud
Validation: 90% non-fraud/10% fraud
Test: 90% non-fraud/10% fraud

Data Splits (pt. 2)

We typically want the splits to be stratified. Meaning that each data split maintains the same ratios.

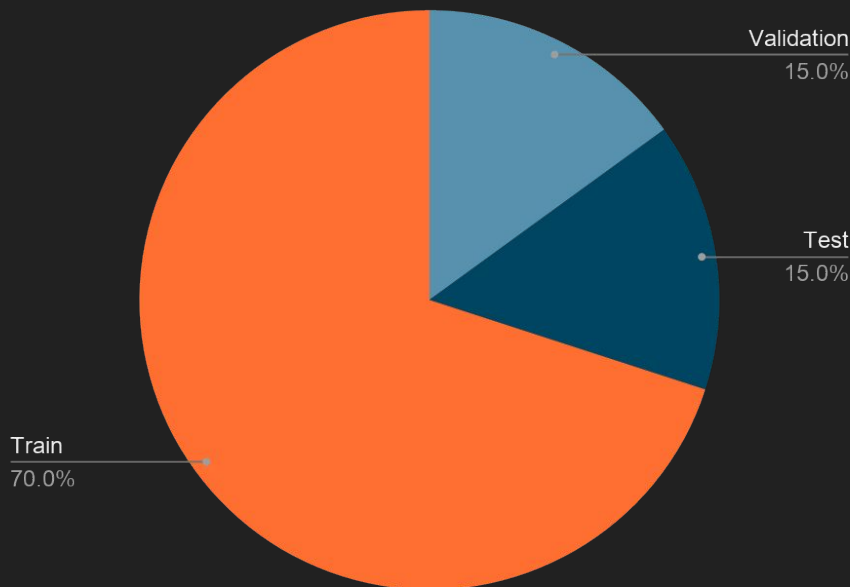
Fraud Example:

Train: 90% non-fraud/ 10% fraud

Validate: 90% non-fraud/ 10% fraud

Test: 90% non-fraud/ 10% fraud

Dataset Splits



Things to watch out for

- Imbalance dataset can skew test results.
 - Fraud detection imbalanced test set example: 95% non-fraud, 5% fraud. If model classifies all as non-fraud, it gets a 95% accuracy. That's pretty good, but we didn't really make anything meaningful.
- Concept Drift
 - Change in relationship of input(x) $\rightarrow$ target(y). Ex. patterns can change over time, so training on historic data may not capture the “now” data.
- Data Drift
 - Change in input(x) distribution. Ex. More high humidity events than low humidity, but relationship of x $\rightarrow$ y doesn't change.
- Label Drift
 - Change in target(y) distribution. Ex. Same weather pattern (inputs), but now more rain (target)

Preprocessing

Preprocessing can help your model learn by making patterns easier to detect.

1. Removing duplicate entries
2. Handling missing values
3. Addressing inconsistencies and outliers
4. Normalization (e.g. zero mean a unit variance)
5. Convert Categorical variables into numbers (one-hot encoding, embeddings)

Preprocessing: Data Augmentation

Data Augmentation: When you have insufficient count of examples (not enough data). You may create synthetic/ artificial data that can mimic the match the “vibe” of your real/actual data.

1. Generating AI images
2. Editing the translation and rotation of the image
3. Use an existing mathematical model (simulator) to generate training examples
4. Add noise to the data to help generalization

What it looks like in Pytorch

```
def build_transform(image_size: int) -> transforms.Compose:
    """Image Preprocessing"""

    return transforms.Compose([
        transforms.Resize(image_size, interpolation=transforms.InterpolationMode.BICUBIC),
        transforms.CenterCrop(image_size),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=(0.485, 0.456, 0.406),
            std=(0.229, 0.224, 0.225),
        ),
    ])
)
```

Key takeaway from this section

- Dataset should be accurate, diverse and relevant to the problem
- Common dataset split 70/15/15 or 60/20/20 into train, validation, and test
- Do not use test set for any decision making
- Careful of dataset imbalance

Training

- How the network learns
- What to optimize for
- How to adjust settings (hyperparameters) to train better

Training Approaches (for Deep Learning)

- **Reinforcement:** Agent (think robot, not necessarily LLM) interact with an environment to perform actions and then receive rewards or penalties as feedback.
 - Ex: Self driving car, AI playing games
- **Supervised:** Learning with features and targets/labeled data
 - Ex: Classifying (Categorizing) dogs vs cats vs birds
- **Unsupervised:** Learning without labeled data, just features.
 - Ex: Predicting rainfall patterns (self-supervised, autoregressive), Grouping into clusters based on similarity

How Does Training (our Neural Network) Work?

1. Weight Initialization (not too relevant for pytorch)
2. For each **epoch** (a complete pass through the training dataset):
 - a. Forward Pass
 - b. Calculate Loss
 - c. Backpropagate
 - d. Optimizer Step

Weight Initialization

Zero: Do not do, network cannot learn because all weights are identical

Random: Random initial values from a gaussian distribution

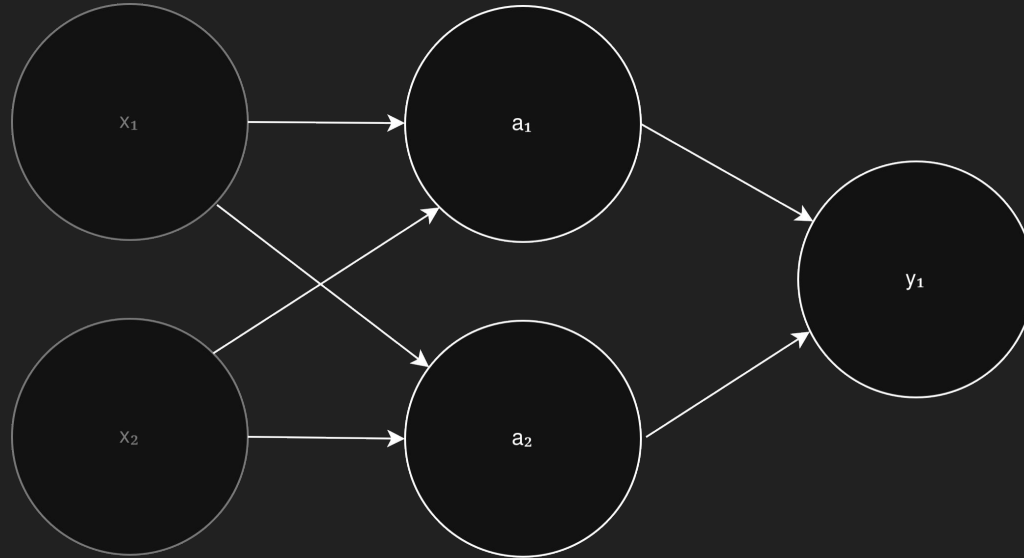
Xavier: Scales random values based on number of inputs and outputs to maintain variance stability, used with sigmoid and tanh typically.

He: Similar to Xavier, but used with ReLU.

Pytorch uses He/Kaiming Uniform by default for linear and convolution if you don't explicitly set anything.

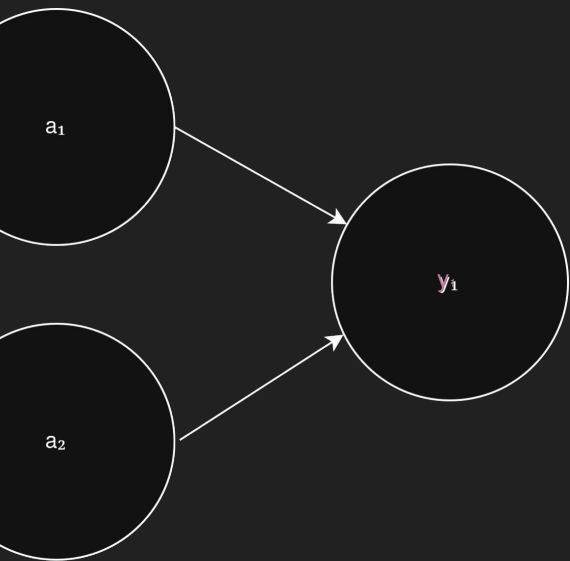
Step 1: Forward Pass

The computation of the neuron we did earlier, the linear transformation and activation functions for each neuron.



Step 2: Calculate Loss

Use a **loss function** to calculate the how off the prediction (y_1) was from the dataset's labels/targets(y).

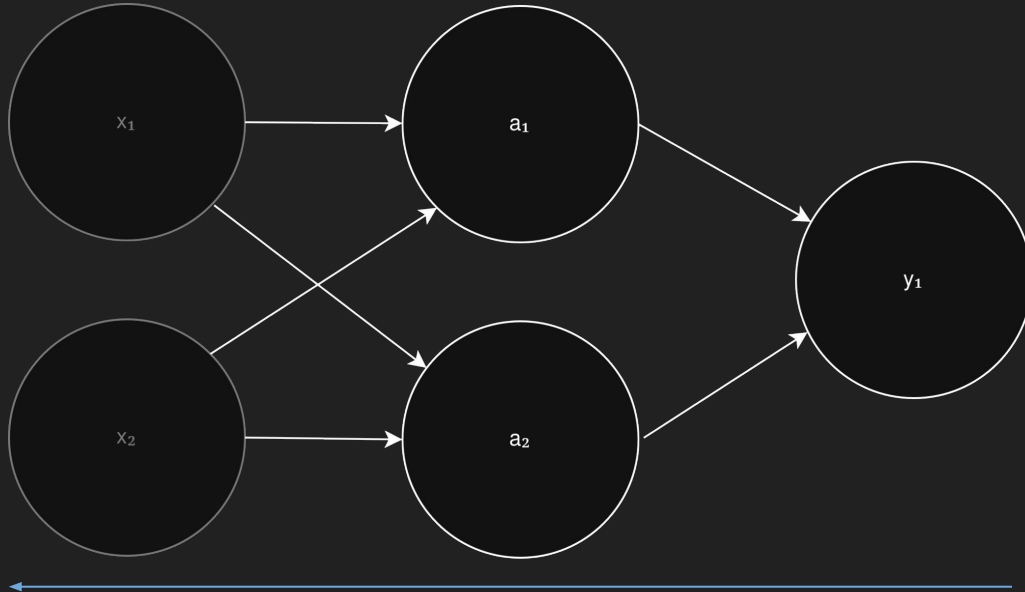


Loss Function	Example Use Case
Binary Cross-Entropy	2 Class Classification
Cross-Entropy	Multiclass Classification
Mean Square Error	Regression
Mean Absolute Error	Regression
...	...

Ask LLM if you don't know which one to use (and there's a lot more loss functions as well)

Step 3: Backward pass/ Backpropagation

Backprop in order to find the **gradients** which are how much the weights and bias of each neuron contributed to the loss.



$$\text{Gradient} = \partial \text{Loss} / \partial \text{weight}$$

We get the gradients which are the partial of the loss with respects to the weights, and we do this for all neurons. We need to use chain rule as well as depth increases.

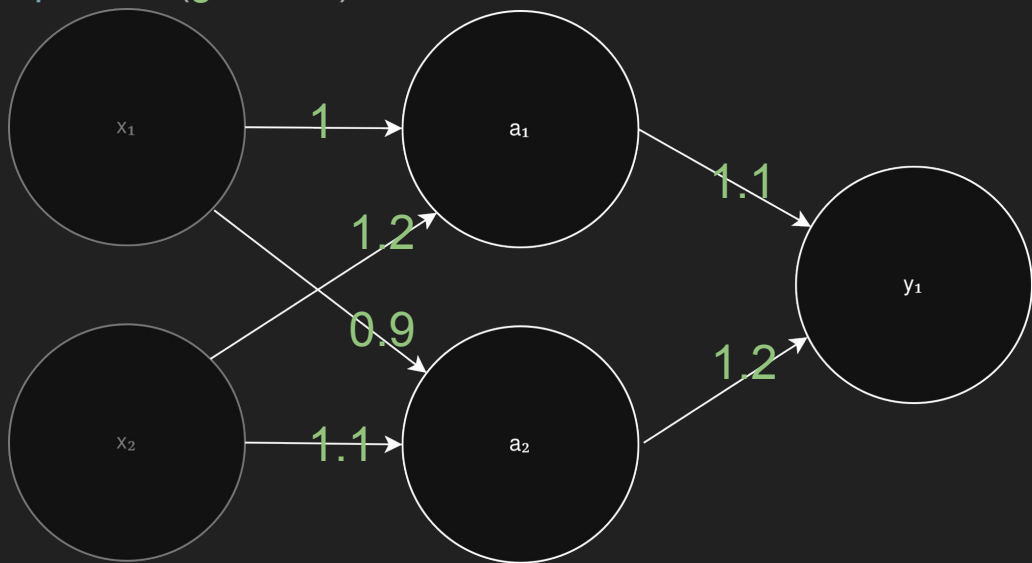
To clarify, we get **1 gradient** per weight or bias.

Step 4: Optimizer Step (Update Weights)

For each weight:

$\text{weight}_{\text{env}} = \text{weight}_{\text{current}} - \text{learning_rate} * \text{optimizer}(\text{gradient})$

$\text{optimizer}(\text{gradient})$



Optimizers are just functions that adjust the **gradient** to nudge the weights in the right direction.

Common Optimizers:

Optimizers	Example Use Case
Adam	General Purpose
AdamW	General, LLM, CNN
SGD+Momentum	CNNs
...	...

*gradients numbers are shown here are for illustrative purposes. Gradients varies based on calculations.

What it looks like in Pytorch

```
for epoch in range(1, EPOCHS + 1):
    model.train()
    running = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()          # clear last batch's gradients (they accumulate)

        # Forward Pass
        logits = model(images)          # raw per-class scores (softmax to get probabilities)
        loss = criterion(logits, labels) # the loss: one scalar to represent how wrong the model is

        # Backprop
        loss.backward()                 # backprop: from the one loss scalar, w/ gradients to all weights
        optimizer.step()                # nudge each weight opposite its gradient
        running += loss.item() * labels.size(0) # sum loss over samples (xN) for this batch

    train_loss = running / len(train_loader.dataset)
    val_acc = evaluate(model, val_loader, device)
    print(f"epoch {epoch:3d}  train_loss {train_loss:.4f}  val_acc {val_acc:.4f}")
```

Common Metrics

How well is our model doing?

- Classification:
 - Accuracy: percentage correct over everything
 - Precision: of all the predicted positives, how many were correct?
 - Recall: of all the actual positives, how many were correctly predicted as positive?
 - F1 Score: harmonic mean of Precision and Recall. F1 is high only when both precision and recall are reasonably high.
- Regression:
 - RMSE (root mean square error): On average, how far off are predictions?
 - MAE (mean absolute error): On average, how far off are predictions? (robust to outliers)
 - R^2 (coefficient of determination): How much of the variance in the target does the model explain, compared to just guessing the average every time?

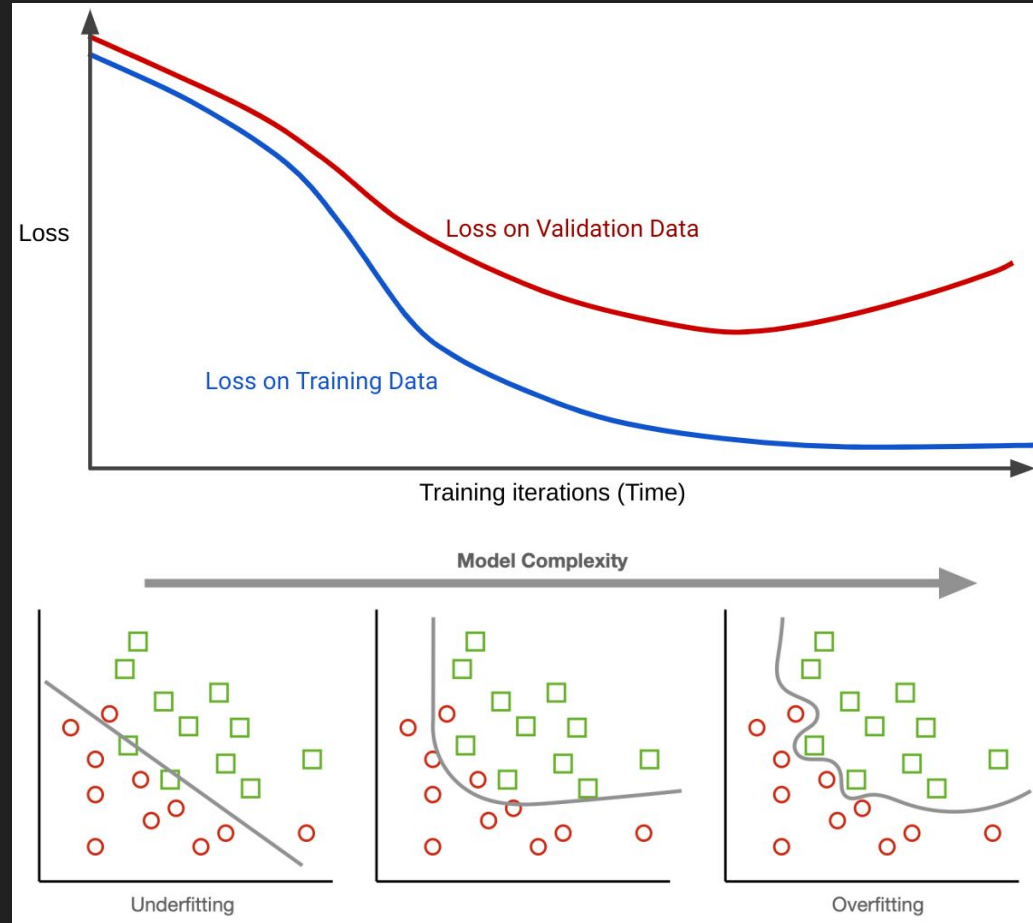
Learning

Okay, how do we know when to stop?

For most loss functions, check the training loss versus the validation loss, the point of divergence suggests onset of overfitting.

Early stopping is used to prevent overfitting by halting when validation loss or accuracy stops improving for a set number of iterations

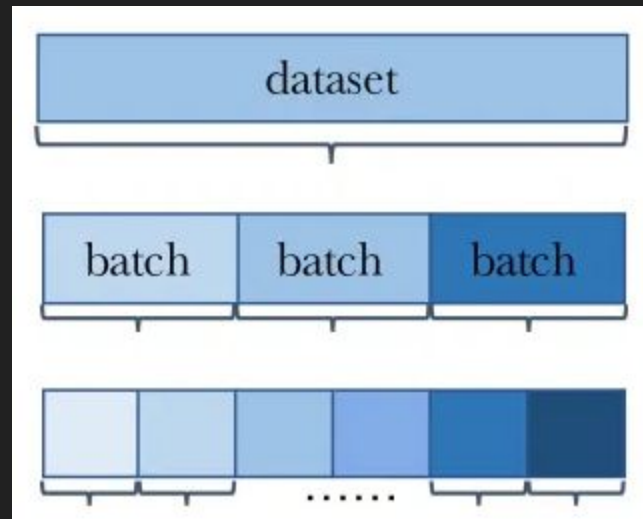
Regularization is any technique that discourages the model from overfitting to the training data



Mini Batching

For each epoch, shuffle and split the dataset into smaller chunks (batches) then update model's the weights after each batch.

- **Benefits:**
 - Memory efficient (not loading entire dataset into memory at once)
 - Acts as regularization
 - Faster convergence when training
- **Disadvantages:**
 - Smaller batches tend to give noisy gradient estimates and reduces learning stability
 - Batch size becomes another hyperparameter



<https://medium.com/@sim30217/mini-batch-baf254a81c68>

Hyperparameters

How can we improve performance (get a better score)?

Hyperparameters are tunable values to adjust learning behaviour.

- Examples:
 - Learning Rate
 - Width/Depth of Network
 - Epochs
 - Optimizer
 - Batch Size (for mini batching)
 - etc.

Hyperparameters: Automation

Ways to automate the search?

- Random Search: Samples random combinations of parameters
- Grid Search: Exhaustively attempts every combination of parameters
- Bayesian Optimization: builds a probabilistic model to choose the optimal next set of parameters to try
- etc.

Key takeaway from this section

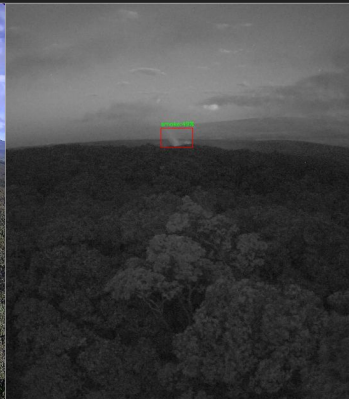
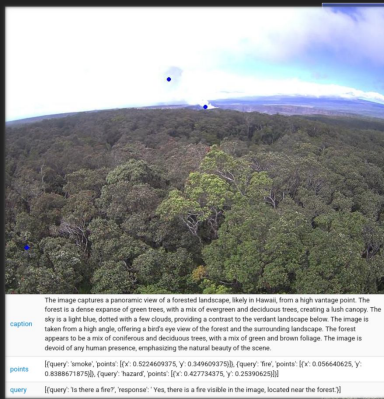
- What the training loop looks like
- Metrics used to describe performance like accuracy
- When to stop training
- Automated hyperparameter tuning

Anomaly Detection

and Hawaii Team's work

Anomaly Detection in Hawaii

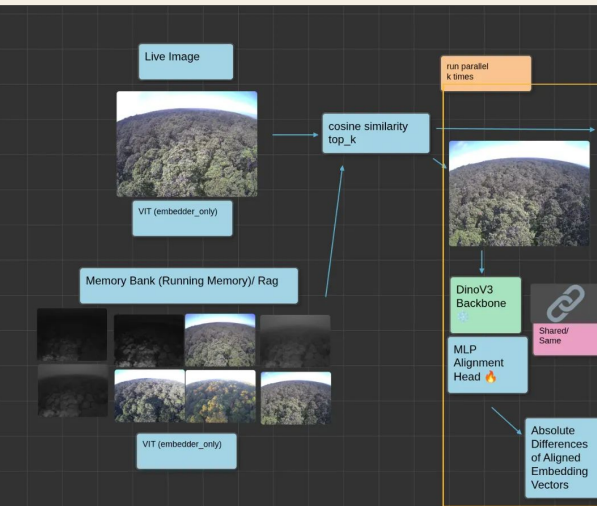
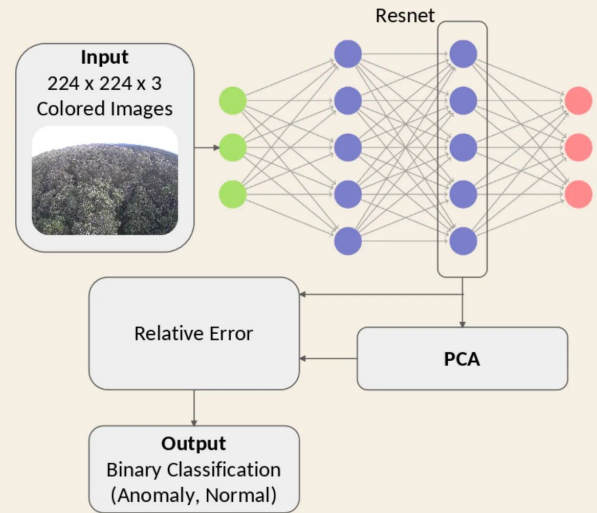
For the Hawaii Team, we currently have a YOLO fire and smoke detection model and a VLM running on the Nodes detecting for wildfires.



Anomaly Detection

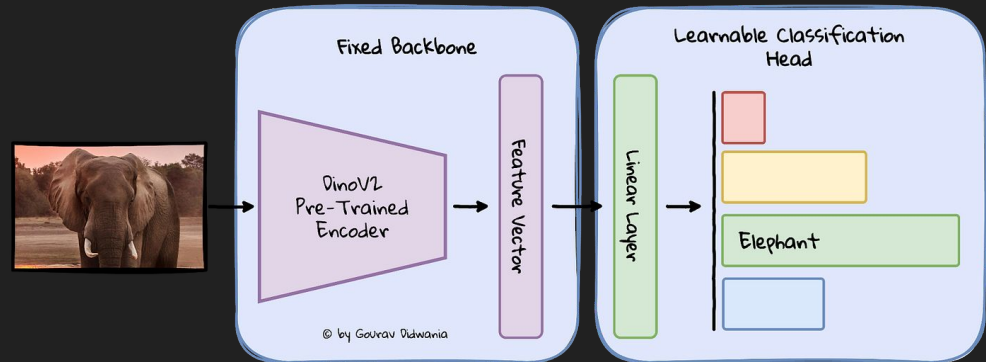
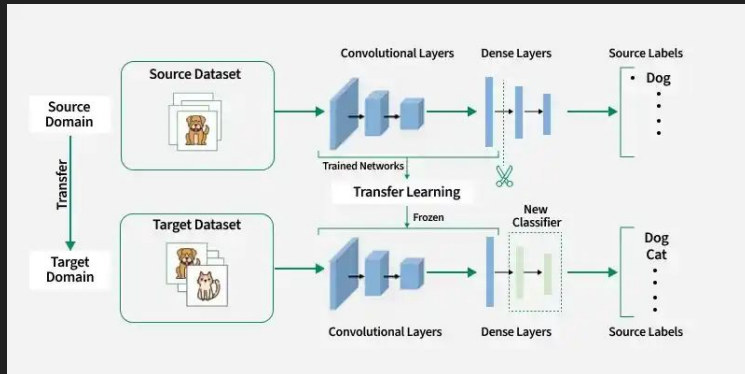
Purely smoke and fire detection is nice but... we hope to generalize and detect for anomalies.

I'm developing a solution that can continuously learn the changing environment and detect for abnormalities depending on what its seen before.



How I approach my work

- Start with a smaller making a smaller model, then go up to larger models when needed
- Pick techniques that can address the problem
 - External memory banks to handle data drifts
- Utilized other people's works or ideas
 - Use foundation models/ fine tuning/ transfer learning



DINOv2 Demo

<https://github.com/cleeeuh/sage-grande-hackathon-dino>